

Рекурсия и структуры

Повторение. Стековый фрейм

Стековый фрейм – область стека, занятая вызовом функции.

За фрейм отвечают два регистра: **EBP**, **ESP**.

В **cdecl calling convention** аргументы передаются через стек. Стек очищает **caller**.

Повторение. Стековый фрейм

Пусть есть функция. Как выглядят ее пролог и эпилог? Как выглядит вызов?

```
1 void swap(int* lhs, int* rhs) {  
2     int tmp = *lhs;  
3     *lhs = *rhs;  
4     *rhs = tmp;  
5 }
```

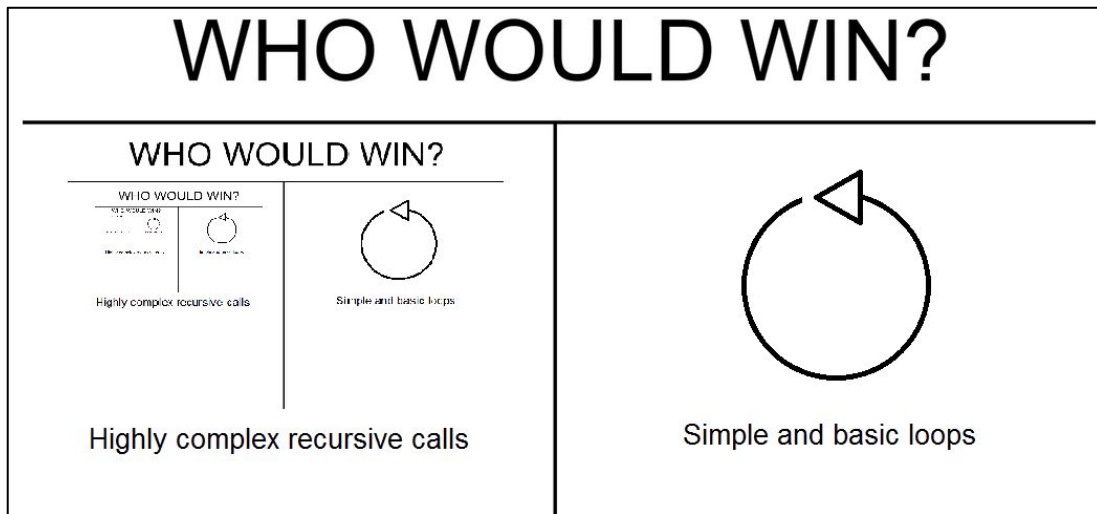
Повторение. Стековый фрейм

```
1 swap:
2   push ebp
3   mov  ebp, esp
4   sub  esp, 16
5
6   # ...
7
8   # mov  ebp, esp
9   # pop  ebp
10  leave
11
12  ret
```

```
1   ; swap(&a, &b);
2   lea  eax, [ebp-16] ; eax <- &b
3   push eax           ; push &b
4   lea  eax, [ebp-20] ; eax <- &a
5   push eax           ; push &a
6   call swap          ; calling swap
7   add  esp, 8 ; cleaning the stack
```

Хвостовая рекурсия и стековый фрейм

- Мы уже видели, что стековый фрейм решает проблему рекурсивных вызовов: локальные переменные каждый раз выделяются в новой области стека.
- Но всегда ли он нужен?



Хвостовая рекурсия и стековый фрейм

- Мы уже видели, что стековый фрейм решает проблему рекурсивных вызовов: локальные переменные каждый раз выделяются в новой области стека.
- Но всегда ли он нужен?
- Что если у нас не будет локальных переменных?

Хвостовая рекурсия

Рассмотрим частный случай рекурсивной функции, в котором вызов функции – последняя операция. То есть мы никогда не вернемся в вызывающую функцию. Такая рекурсия называется хвостовой

```
1 void tail_recursion(int x) {  
2     if (x <= 0)  
3         return;  
4  
5     printf("%d ", x);  
6     return tail_recursion(x - 1);  
7 }
```

Хвостовая рекурсия

Является ли эта функция хвостовой рекурсией?

```
1 int fact(int n) {  
2     if (n <= 1)  
3         return 1;  
4  
5     return n * fact(n - 1);  
6 }
```


Хвостовая рекурсия

Нет, не является:

- Сначала мы сделаем рекурсивный вызов **fact(n - 1)**
- Затем вернемся на уровень выше и умножим на **n**
- Как сделать такую рекурсию хвостовой?

```
1 int fact(int n) {  
2     if (n <= 1)  
3         return 1;  
4  
5     return n * fact(n - 1);  
6 }
```

Хвостовая рекурсия

- Как сделать такую рекурсию хвостовой?
- Воспользовавшись параметром-аккумулятором, можем сделать рекурсию хвостовой

```
1 int fact_tr(int n, int acc) {  
2     if (n <= 1)  
3         return acc;  
4     return fact_tr(n - 1, acc * n);  
5 }  
6
```

Хвостовая рекурсия

Соберем все вместе:

- Иногда рекурсию можно просто переписать как хвостовую (строго говоря – всегда, но в некоторых случаях потребуется завести свой программный стек).
- При хвостовой рекурсии нам не нужен стековый фрейм – все локальные переменные можно спокойно переписать. Компилятор такое любит :)

Хвостовая рекурсия

Обычный факториал

```
1 int fact(int n) {  
2     int res = n;  
3  
4     if (n <= 1)  
5         return 1;  
6  
7     res *= fact(n - 1);  
8     return res;  
9 }
```

<https://godbolt.org/z/bYvMfvjWe>

```
1 fact:  
2     push    ebp  
3     mov     ebp, esp  
4     sub     esp, 4  
5     mov     eax, DWORD PTR [ebp+8]  
6     mov     DWORD PTR [ebp-4], eax  
7     cmp     DWORD PTR [ebp+8], 1  
8     jg      .L2  
9     mov     eax, 1  
10    jmp     .L3  
11 .L2:  
12    mov     eax, DWORD PTR [ebp+8]  
13    sub     eax, 1  
14    push    eax  
15    call    fact  
16    add     esp, 4  
17    mov     edx, DWORD PTR [ebp-4]  
18    imul    eax, edx  
19    mov     DWORD PTR [ebp-4], eax  
20    mov     eax, DWORD PTR [ebp-4]  
21 .L3:  
22    leave  
23    ret
```

Хвостовая рекурсия

Обычный факториал

```
1 int fact(int n) {  
2     int res = n;  
3  
4     if (n <= 1)  
5         return 1;  
6  
7     res *= fact(n - 1);  
8     return res;  
9 }
```

Выделяем фрейм под res

Пушим аргументы для вызова

Очистка аргументов

```
1 fact:  
2     push    ebp  
3     mov     ebp, esp  
4     sub     esp, 4  
5     mov     eax, DWORD PTR [ebp+8]  
6     mov     DWORD PTR [ebp-4], eax  
7     cmp     DWORD PTR [ebp+8], 1  
8     jg      .L2  
9     mov     eax, 1  
10    jmp     .L3  
11 .L2:  
12    mov     eax, DWORD PTR [ebp+8]  
13    sub     eax, 1  
14    push    eax  
15    call    fact  
16    add     esp, 4  
17    mov     edx, DWORD PTR [ebp-4]  
18    imul    eax, edx  
19    mov     DWORD PTR [ebp-4], eax  
20    mov     eax, DWORD PTR [ebp-4]  
21 .L3:  
22    leave  
23    ret
```

Хвостовая рекурсия

Факториал через хвостовую рекурсию

```
12 int fact_tr(int n, int acc) {  
13     int next_n = n - 1;  
14     int next_acc = acc * n;  
15  
16     if (n <= 1)  
17         return acc;  
18  
19     return fact_tr(next_n, next_acc);  
20 }
```

<https://godbolt.org/z/bYvMfvjWe>

```
24 fact_tr:  
25     push    ebp  
26     mov     ebp, esp  
27     sub     esp, 8  
28     mov     eax, DWORD PTR [ebp+8]  
29     mov     edx, DWORD PTR [ebp+12]  
30     lea     ecx, [eax-1]  
31     mov     DWORD PTR [ebp-4], ecx  
32     mov     ecx, edx  
33     imul    ecx, eax  
34     mov     DWORD PTR [ebp-8], ecx  
35     cmp     eax, 1  
36     jg      .L5  
37     mov     eax, edx  
38     jmp     .L7  
39 .L5:  
40     mov     eax, DWORD PTR [ebp-8]  
41     mov     DWORD PTR [ebp+12], eax  
42     mov     eax, DWORD PTR [ebp-4]  
43     mov     DWORD PTR [ebp+8], eax  
44     leave  
45     jmp     fact_tr  
46 .L7:  
47     leave  
48     ret
```

Хвостовая рекурсия

Факториал через хвостовую рекурсию

Создаем новый фрейм для next_n, next_acc
Обращаемся “наверх” за аргументами

```
12 int fact_tr(int n, int acc) {  
13     int next_n = n - 1;  
14     int next_acc = acc * n;  
15  
16     if (n <= 1)  
17         return acc;  
18  
19     return fact_tr(next_n, next_acc);  
20 }
```

Кладем новые аргументы вызова на стек

Завершаем текущий фрейм
Делаем обычный jmp, т.е. переписываем тот же фрейм

```
24 fact_tr:  
25     push    ebp  
26     mov     ebp, esp  
27     sub     esp, 8  
28     mov     eax, DWORD PTR [ebp+8]  
29     mov     edx, DWORD PTR [ebp+12]  
30     lea     ecx, [eax-1]  
31     mov     DWORD PTR [ebp-4], ecx  
32     mov     ecx, edx  
33     imul    ecx, eax  
34     mov     DWORD PTR [ebp-8], ecx  
35     cmp     eax, 1  
36     jg      .L5  
37     mov     eax, edx  
38     jmp     .L7  
39 .L5:  
40     mov     eax, DWORD PTR [ebp-8]  
41     mov     DWORD PTR [ebp+12], eax  
42     mov     eax, DWORD PTR [ebp-4]  
43     mov     DWORD PTR [ebp+8], eax  
44     leave  
45     jmp     fact_tr  
46 .L7:  
47     leave  
48     ret
```

<https://godbolt.org/z/bYvMfvjWe>

Структуры в функциях

Мы уже поговорили, что аргументы передаются через стек, а возвращаются через регистр EAX.

Но что если у нас такая функция?

Тут два вопроса:

- Как передать аргументы?
- Как вернуть структуру?

```
1  
1  typedef struct {
2      int x;
3      int y;
4  } Vec2;
5
6  Vec2 add(Vec2 v1, Vec2 v2) {
7      Vec2 result = {
8          .x = v1.x + v2.x,
9          .y = v1.y + v2.y
10     };
11     return result;
12 }
```


Структуры в функциях

С передачей все просто:

- Структура – это просто набор байт. Они передаются в функцию через стек.
- То есть структура “линеаризуется” и поля передаются как отдельные “аргументы”.
- В итоге в асме мы увидим обычную функцию 4х аргументов, так?

```
1    
1  typedef struct {  
2      int x;  
3      int y;  
4  } Vec2;  
5  
6  Vec2 add(Vec2 v1, Vec2 v2) {  
7      Vec2 result = {  
8          .x = v1.x + v2.x,  
9          .y = v1.y + v2.y  
10     };  
11     return result;  
12 }
```

Структуры в функциях

В итоге в асме мы увидим обычную функцию 4х аргументов, так?

```
7  Vec2 add(Vec2 v1, Vec2 v2) {  
8      Vec2 result = {  
9          .x = v1.x + v2.x,  
10         .y = v1.y + v2.y  
11     };  
12     return result;  
13 }
```

<https://godbolt.org/z/a1x1nzKeq>

```
1  add:  
2      push    ebp  
3      mov     ebp, esp  
4      sub     esp, 8  
5      mov     edx, DWORD PTR [ebp+12]  
6      mov     eax, DWORD PTR [ebp+20]  
7      add     eax, edx  
8      mov     DWORD PTR [ebp-8], eax  
9      mov     edx, DWORD PTR [ebp+16]  
10     mov     eax, DWORD PTR [ebp+24]  
11     add     eax, edx  
12     mov     DWORD PTR [ebp-4], eax  
13     mov     ecx, DWORD PTR [ebp+8]  
14     mov     eax, DWORD PTR [ebp-8]  
15     mov     edx, DWORD PTR [ebp-4]  
16     mov     DWORD PTR [ecx], eax  
17     mov     DWORD PTR [ecx+4], edx  
18     mov     eax, DWORD PTR [ebp+8]  
19     leave   4  
20     ret     4
```

Структуры в функциях

В итоге в асме мы увидим обычную
функцию 4х аргументов

```
7  Vec2 add(Vec2 v1, Vec2 v2)
8      Vec2 result;
9      result.x = v1.x + v2.x;
10     result.y = v1.y + v2.y;
11     };
12     return result;
13 }
```

edx <- v1.x
eax <- v2.x
складываем

кладем результат в result.x

то же самое для .y

знакомьтесь: ret с аргументом
очищает 4 байта стека, чтоб этого не делал
caller

```
1  add:
2      push    ebp
3      mov     ebp, esp
4      sub     esp, 8
5      mov     edx, DWORD PTR [ebp+12]
6      mov     eax, DWORD PTR [ebp+20]
7      add     eax, edx
8      mov     DWORD PTR [ebp-8], eax
9      mov     edx, DWORD PTR [ebp+16]
10     mov     eax, DWORD PTR [ebp+24]
11     add     eax, edx
12     mov     DWORD PTR [ebp-4], eax
13     mov     ecx, DWORD PTR [ebp+8]
14     mov     eax, DWORD PTR [ebp-8]
15     mov     edx, DWORD PTR [ebp-4]
16     mov     DWORD PTR [ecx], eax
17     mov     DWORD PTR [ecx+4], edx
18     mov     eax, DWORD PTR [ebp+8]
19     leave   4
20     ret
```

Структуры в функциях

В итоге в асме мы увидим обычную функцию 4х аргументов, так?

```
7  Vec2 add(Vec2 v1, Vec2 v2) {  
8      Vec2 result = {  
9          .x = v1.x + v2.x,  
10         .y = v1.y + v2.y  
11     };  
12     return result;  
13 }
```

что это такое??

```
1  add:  
2      push    ebp  
3      mov     ebp, esp  
4      sub     esp, 8  
5      mov     edx, DWORD PTR [ebp+12]  
6      mov     eax, DWORD PTR [ebp+20]  
7      add     eax, edx  
8      mov     DWORD PTR [ebp-8], eax  
9      mov     edx, DWORD PTR [ebp+16]  
10     mov     eax, DWORD PTR [ebp+24]  
11     add     eax, edx  
12     mov     DWORD PTR [ebp-4], eax  
13     mov     ecx, DWORD PTR [ebp+8]  
14     mov     eax, DWORD PTR [ebp-8]  
15     mov     edx, DWORD PTR [ebp-4]  
16     mov     DWORD PTR [ecx], eax  
17     mov     DWORD PTR [ecx+4], edx  
18     mov     eax, DWORD PTR [ebp+8]  
19     leave   4  
20     ret     4
```

Структуры в функциях

По положительным смещениям еbp идут
аргументы:

$ebp + 24 \leftarrow v2.y$
 $ebp + 20 \leftarrow v2.x$
 $ebp + 12 \leftarrow v1.y$
 $ebp + 16 \leftarrow v1.x$

То есть $ebp + 8$ – какой-то секретный 0й аргумент?
Тогда выходит:

$ecx \leftarrow$ секретный 0й аргумент
 $eax \leftarrow result.y$
 $edx \leftarrow result.x$

по указателю 0го аргумента $\leftarrow$ eax
по указателю 0го аргумента $+ 4 \leftarrow$ edx
в eax кладем 0й аргумент

```
1  add:
2      push    ebp
3      mov     ebp, esp
4      sub     esp, 8
5      mov     edx, DWORD PTR [ebp+12]
6      mov     eax, DWORD PTR [ebp+20]
7      add     eax, edx
8      mov     DWORD PTR [ebp-8], eax
9      mov     edx, DWORD PTR [ebp+16]
10     mov     eax, DWORD PTR [ebp+24]
11     add     eax, edx
12     mov     DWORD PTR [ebp-4], eax
13     mov     ecx, DWORD PTR [ebp+8]
14     mov     eax, DWORD PTR [ebp-8]
15     mov     edx, DWORD PTR [ebp-4]
16     mov     DWORD PTR [ecx], eax
17     mov     DWORD PTR [ecx+4], edx
18     mov     eax, DWORD PTR [ebp+8]
19     leave
20     ret     4
```

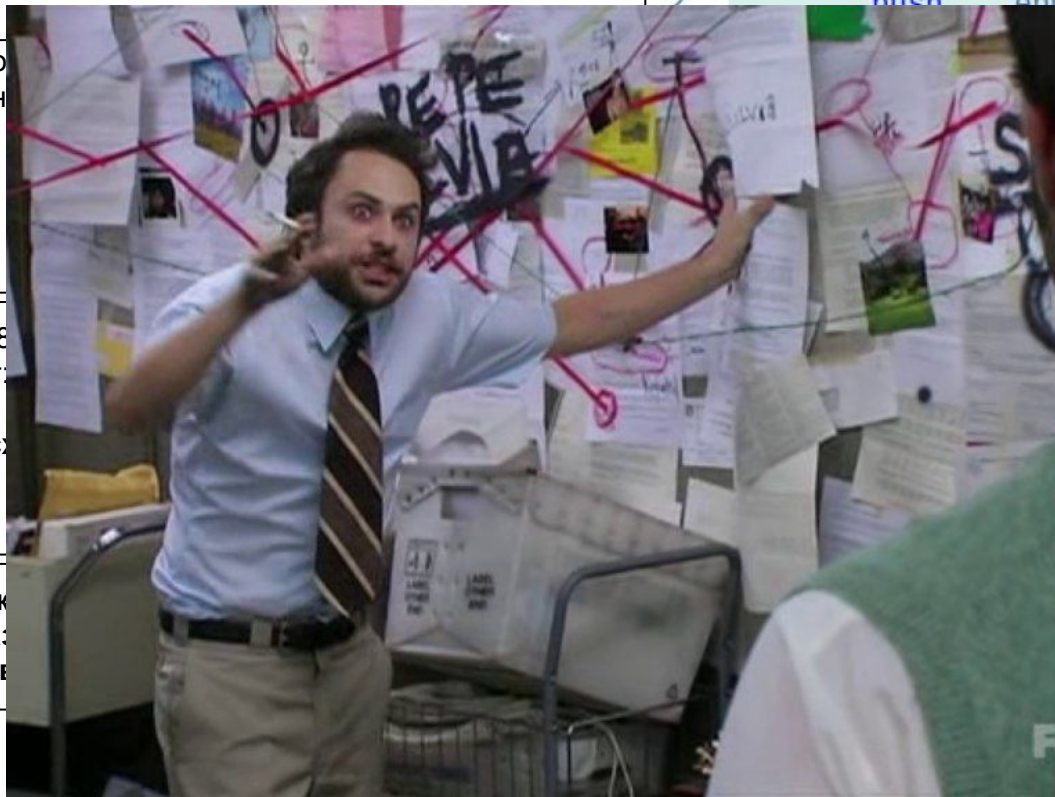

Структуры в функциях

По поло
аргумент

То есть `ebp + 8`
Тогда выходит

если

по ук
по указ



```
1 add:
```

```
2 push ebp
```

```
, esp
```

```
8
```

```
, DWORD PTR [ebp+12]
```

```
, DWORD PTR [ebp+20]
```

```
, edx
```

```
RD PTR [ebp-8], eax
```

```
, DWORD PTR [ebp+16]
```

```
, DWORD PTR [ebp+24]
```

```
, edx
```

```
RD PTR [ebp-4], eax
```

```
, DWORD PTR [ebp+8]
```

```
, DWORD PTR [ebp-8]
```

```
, DWORD PTR [ebp-4]
```

```
RD PTR [ecx], eax
```

```
RD PTR [ecx+4], edx
```

```
, DWORD PTR [ebp+8]
```

Структуры в функциях

То есть `ebp + 8` – какой-то секретный 0й аргумент?
Тогда выходит:

`ecx <- секретный 0й аргумент`
`eax <- result.y`
`edx <- result.x`

по указателю 0го аргумента <- `eax`
по указателю 0го аргумента + 4 <- `eax`
в `eax` кладем 0й аргумент

Применяя навыки **дедукции**, понимаем:

1. 0й аргумент – это указатель, раз он разыменовывается через `[]`
2. Он указывает на структуру, раз в него кладутся `x` и `y`
3. Возвращается именно этот указатель

```
1  add:
2      push    ebp
3      mov     ebp, esp
4      sub     esp, 8
5      mov     edx, DWORD PTR [ebp+12]
6      mov     eax, DWORD PTR [ebp+20]
7      add     eax, edx
8      mov     DWORD PTR [ebp-8], eax
9      mov     edx, DWORD PTR [ebp+16]
10     mov     eax, DWORD PTR [ebp+24]
11     add     eax, edx
12     mov     DWORD PTR [ebp-4], eax
13     mov     ecx, DWORD PTR [ebp+8]
14     mov     eax, DWORD PTR [ebp-8]
15     mov     edx, DWORD PTR [ebp-4]
16     mov     DWORD PTR [ecx], eax
17     mov     DWORD PTR [ecx+4], edx
18     mov     eax, DWORD PTR [ebp+8]
19     leave
20     ret     4
```

Структуры в функциях

Применяя навыки дедукции, понимаем:

1. Ой аргумент – это указатель, раз он разыменовывается через []
2. Он указывает на структуру, раз в него кладутся *x* и *y*
3. Возвращается именно этот указатель

```
11 void add(Vec2 *out, Vec2 v1, Vec2 v2) {  
10     Vec2 result = {  
9         .x = v1.x + v2.x,  
8         .y = v1.y + v2.y  
7     };  
6  
5     out->x = result.x;  
4     out->y = result.y;  
3  
2     return;  
1 }
```

```
1  add:  
2      push    ebp  
3      mov     ebp, esp  
4      sub     esp, 8  
5      mov     edx, DWORD PTR [ebp+12]  
6      mov     eax, DWORD PTR [ebp+20]  
7      add     eax, edx  
8      mov     DWORD PTR [ebp-8], eax  
9      mov     edx, DWORD PTR [ebp+16]  
10     mov     eax, DWORD PTR [ebp+24]  
11     add     eax, edx  
12     mov     DWORD PTR [ebp-4], eax  
13     mov     ecx, DWORD PTR [ebp+8]  
14     mov     eax, DWORD PTR [ebp-8]  
15     mov     edx, DWORD PTR [ebp-4]  
16     mov     DWORD PTR [ecx], eax  
17     mov     DWORD PTR [ecx+4], edx  
18     mov     eax, DWORD PTR [ebp+8]  
19     leave  
20     ret     4
```


Структуры в функциях

Но кто же выделяет для нас этот 0й out-параметр. Давайте посмотрим на вызов

```
15 int main() {  
16     Vec2 a = {10, 20};  
17     Vec2 b = {30, 40};  
18  
19     Vec2 sum = add(a, b);  
20  
21 }
```

```
21 main:  
22     push    ebp  
23     mov     ebp, esp  
24     sub     esp, 24  
25     mov     DWORD PTR [ebp-8], 10  
26     mov     DWORD PTR [ebp-4], 20  
27     mov     DWORD PTR [ebp-16], 30  
28     mov     DWORD PTR [ebp-12], 40  
29     lea     eax, [ebp-24]  
30     push    DWORD PTR [ebp-12]  
31     push    DWORD PTR [ebp-16]  
32     push    DWORD PTR [ebp-4]  
33     push    DWORD PTR [ebp-8]  
34     push    eax  
35     call    add  
36     add     esp, 16  
37     mov     eax, 0  
38     leave  
39     ret
```

Структуры в функциях

Но кто же выделяет для нас этот 0й out-параметр. Давайте посмотрим на ВЫЗОВ

```
15 int main()  
16     Vec2 a  
17     Vec2 b  
18  
19     Vec2 s  
20  
21 }
```

Заводим локальные переменные
а и b

На стеке они выглядят как “линеаризованные”
структуры

Выделяем память под возвращаемую структуру

Пушим аргументы функции

Секретный 0й аргумент!

```
21 main:
```

```
22     push    ebp  
23     mov     ebp, esp  
24     sub     esp, 24
```

```
mov     DWORD PTR [ebp-8], 10  
mov     DWORD PTR [ebp-4], 20  
mov     DWORD PTR [ebp-16], 30  
mov     DWORD PTR [ebp-12], 40
```

```
lea     eax, [ebp-24]
```

```
push    DWORD PTR [ebp-12]  
push    DWORD PTR [ebp-16]  
push    DWORD PTR [ebp-4]  
push    DWORD PTR [ebp-8]
```

```
push    eax
```

```
35     call    add  
36     add     esp, 16  
37     mov     eax, 0  
38     leave  
39     ret
```

Структуры в функциях

Итого, компилятор перед вызовом функции:

1. выделяет место в стековом фрейме caller
2. передает callee указатель на это место
3. Код callee генерируется так, чтоб записать результат в это место
4. Указатель на это место возвращается из callee в **eax**

```
21  main:
22      push    ebp
23      mov     ebp, esp
24      sub     esp, 24
25      mov     DWORD PTR [ebp-8], 10
26      mov     DWORD PTR [ebp-4], 20
27      mov     DWORD PTR [ebp-16], 30
28      mov     DWORD PTR [ebp-12], 40
29      lea     eax, [ebp-24]
30      push    DWORD PTR [ebp-12]
31      push    DWORD PTR [ebp-16]
32      push    DWORD PTR [ebp-4]
33      push    DWORD PTR [ebp-8]
34      push    eax
35      call    add
36      add     esp, 16
37      mov     eax, 0
38      leave
39      ret
```

Структуры в функциях

Пара общих слов:

Писать код как на слайде – плохая практика. Стек – медленная штука по сравнению с регистрами.

Помните про System V ABI: там первые 6 аргументов разложены по регистрам.

Компилятор постарается разложить вашу структуру в регистры, но не факт что у него получится

```
1  
1  typedef struct {
2      int x;
3      int y;
4  } Vec2;
5
6  Vec2 add(Vec2 v1, Vec2 v2) {
7      Vec2 result = {
8          .x = v1.x + v2.x,
9          .y = v1.y + v2.y
10     };
11     return result;
12 }
```

Структуры в функциях

Компилятор постарается разложить вашу структуру в регистры, но не факт что у него получится

Поэтому структуры (классы) лучше всего передавать в функцию по указателю (ссылке).

Тогда компилятор положит эти указатели в регистры и ваш код станет быстрее

```
1  
2  typedef struct {
3      int x;
4      int y;
5  } Vec2;
6  Vec2 add(Vec2 v1, Vec2 v2) {
7      Vec2 result = {
8          .x = v1.x + v2.x,
9          .y = v1.y + v2.y
10     };
11     return result;
12 }
```

Выравнивание данных

Давайте вспомним:

- Зачем нужно выравнивание?
- Как будут выровнены поля в **S**?
- Как будет выглядеть обращение к **b** на 9-10 строках?

<https://godbolt.org/z/vzWM18nG9>

```
1  typedef struct {  
2      char a;  
3      int b;  
4      char c;  
5  } S;  
6  int sum_b(S *arr, int n) {  
7      int s = 0;  
8      for (int i = 0; i < n; i++) {  
9          S arr_i = arr[i];  
10         s += arr_i.b;  
11     }  
12     return s;  
13 }  
14  
15 int main() {  
16     S array[2] = {{1, 2, 3}, {4, 5, 6}};  
17     int res = sum_b(array, 2);  
18  
19     printf("%d\n", res);  
20 }
```

Рекомендуемая литература

- Bryant, R. E., & O'Hallaron, D. R. (2016). Computer Systems: A Programmer's Perspective (3rd ed.). Pearson. – **глава 3.9 стр 271 — 279**